# LA Food Scout

A Machine Learning App for Discovering Highly Rated Restaurants in Los Angeles

Zhixian Wang | STAT 418 | June 2, 2026

| Live App | https://la-food-scout-app-886054929408.us-central1.run.app |
| API Docs | https://la-food-scout-api-886054929408.us-central1.run.app/docs |
| GitHub | https://github.com/ArcherX0X/la-food-scout |

# Project Overview & Motivation

**Problem statement**: LA has 30,000+ restaurants. Google ratings are tightly clustered — 89% of places are ≥4.0★ — so the real signal is ≥4.5★, which only 41% of restaurants reach.

**Question**: Can we predict whether a restaurant is highly rated (≥4.5★) *before* trying it, using only publicly available metadata?

## Why It Matters:

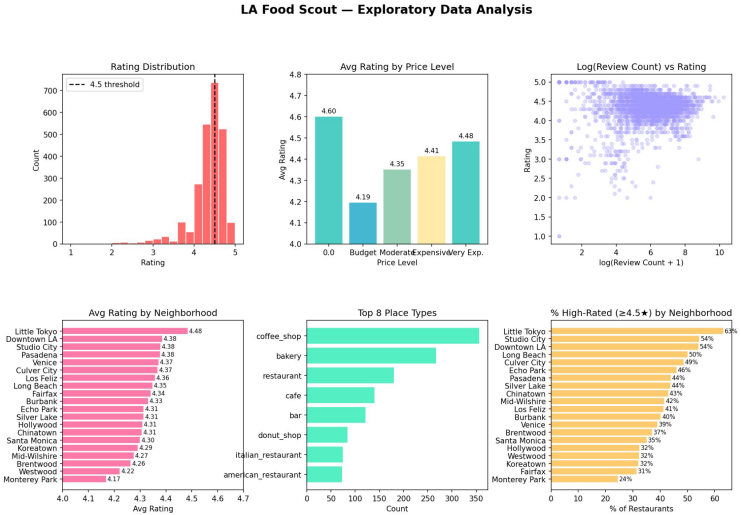
1. Tourists and locals want a quick filter before committing to a spot
2. Restaurant owners can understand what factors correlate with high ratings
3. Demonstrates an end-to-end ML pipeline on real geospatial data

## Approach:

1. Collect data via Google Places API across 20 LA neighborhoods
2. Train a binary classifier on neighborhood, cuisine type, price, and review volume
3. Serve predictions via a FastAPI backend + Streamlit frontend, both on Google Cloud Run

# Data Collection & Preprocessing

| | |
|---|---|
| Source | Google Places API v1 — `searchNearby` endpoint |
| Neighborhoods | 20 across LA (Koreatown, Santa Monica, Downtown LA, Little Tokyo, …) |
| Search strategy | 5 points per neighborhood (center + N/S/E/W at ~1.3 km) |
| Place types queried | `restaurant`, `cafe`, `bakery`, `bar` |
| De-duplication | By Google place ID |
| Final dataset | **2,443 unique restaurants** |




Fields collected: `name`, `rating`, `review_count`, `price_level`, `primary_type`, `lat/lon`, `business_status`, `neighborhood`

Total restaurants | 2,443
High-rated (≥4.5★) | 998 — **40.9%**
Neighborhoods | 20 Unique place types | 144
Median review count | 390
Mean review count | 862 (right-skewed)

# Preprocessing

| Step | Detail |
|---|---|
| Drop missing ratings | 0 rows dropped (API only returns rated places) |
| Impute `price_level` | 637 missing (26%) → filled with median (= 2.0) |
| Drop missing `primary_type` | 3 rows dropped |
| Log-transform reviews | `review_count_log = log1p(review_count)` — raw range 1–28,054; log range 0–10.2 |
| **Binary target** | `high_rated = 1` if `rating ≥ 4.5` |

# Model Development & Evaluation

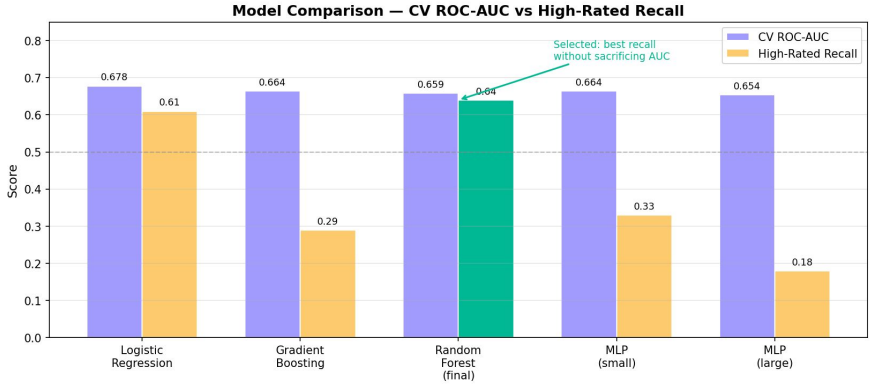
| Model | CV ROC-AUC | Test ROC-AUC | HR Recall | HR Precision | Accuracy |
|---|---|---|---|---|---|
| Logistic Regression | 0.678 ± 0.038 | 0.659 | 0.61 | 0.51 | 0.61 |
| Gradient Boosting | 0.664 ± 0.032 | 0.654 | 0.29 | 0.67 | 0.65 |
| **Random Forest** | **0.659 ± 0.032** | **0.652** | **0.64** | **0.48** | **0.57** |
| MLP — 2 layers | 0.664 ± 0.018 | 0.635 | 0.33 | 0.61 | 0.64 |
| MLP — 3 layers | 0.654 ± 0.024 | 0.642 | 0.18 | 0.64 | 0.62 |

## Feature Importances Top 4

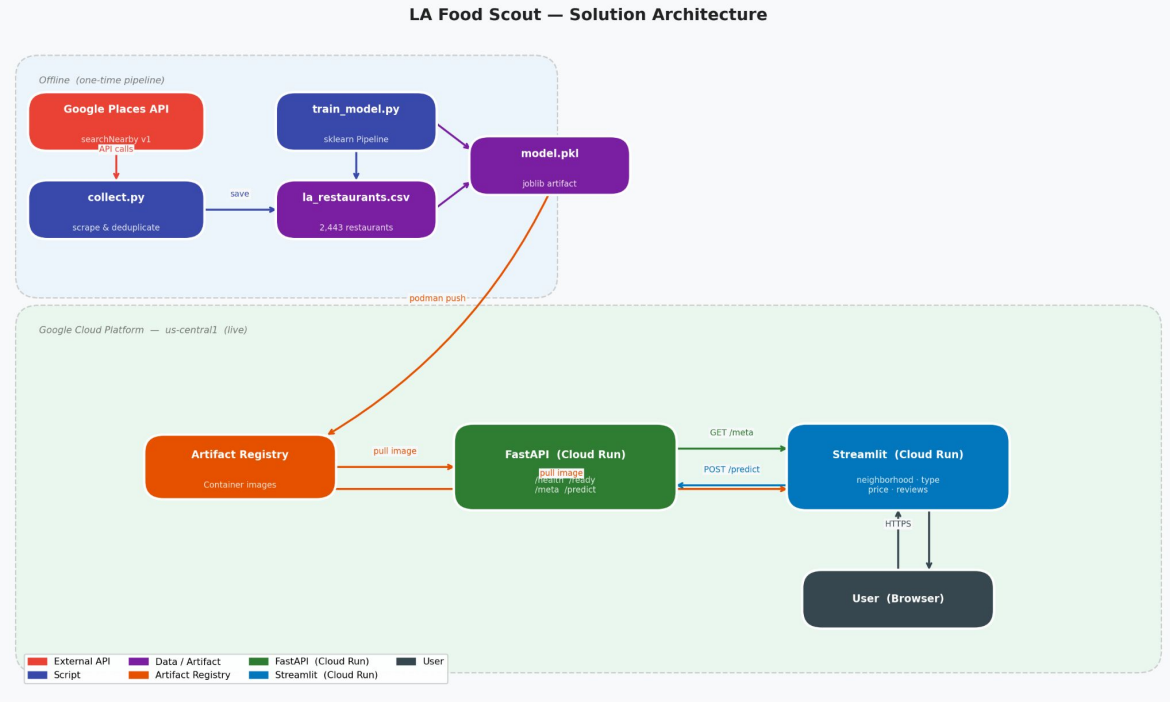
| | |
|---|---|
| primary_type | 63.5% |
| neighborhood | 17.9% |
| review_count_log | 15.0% |
| price_level | 3.6% |

Why Random Forest:
Only model that combined **strong recall** (0.64) with **competitive AUC** (0.652).

Accuracy is misleading under class imbalance — a model predicting 'not high-rated' every time would score 59%.



# Solution Architecture




Model is bundled into the API container — no separate model store needed

`min-instances=0` — scales to zero when idle, stays within GCP free tier

| Layer | Tech | Where |
|---|---|---|
| Data Collection | Google Places API | Local (one-time) |
| Dataset | `la_restaurants.csv` | GitHub repo |
| Model training | scikit-learn Pipeline | Local (one-time) |
| Model artifact | `model.pkl` (joblib) | API container |
| Prediction API | FastAPI Uvicorn | Google Cloud Run |
| Web app | Streamlit | Google Cloud Run |
| Container registry | Google Artifact Reg. | GCP project |
| Container build | Podman linux/amd64 | Local → Registry |

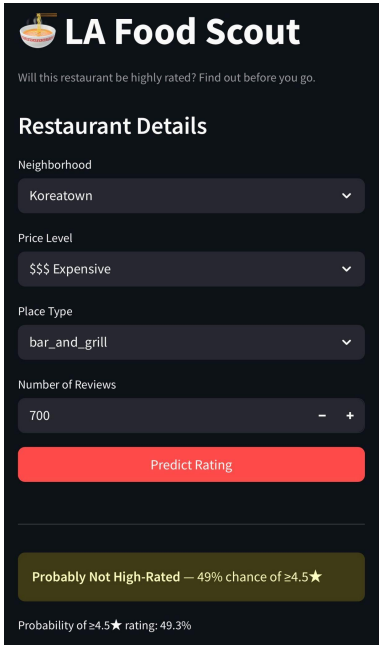
# Application Features

Live Demo

## Streamlit UI

4 inputs, populated live from the API's `/meta` endpoint — no hardcoded lists:

| Input | Options |
|---|---|
| Neighborhood | 20 LA neighborhoods |
| Place Type | 144 place types (italian_restaurant, sushi_restaurant, …) |
| Price Level | $ Budget / $$ Moderate / $$$ Expensive / $$$$ Very Expensive |
| Number of Reviews | Numeric input (default 500) |




Output:
- `st.progress` bar showing probability of ≥4.5★
- Green banner: *"Likely High-Rated ⭐ — XX% confidence"*
- Yellow banner: *"Probably Not High-Rated — XX% chance of ≥4.5★"*

# Challenges, Learnings & Future Work

## Technical Challenges

| | |
|---|---|
| Class imbalance (40% high-rated) | `class_weight="balanced"` — boosted HR recall from 0.29 → 0.64 |
| Apple Silicon → linux/amd64 container builds | `podman build --platform linux/amd64` flag |
| **Google Places API 20-result cap per call** | 5 search points per neighborhood (center + 4 directional) |

## Role of AI Assistants

1. Awesome at debugging
2. Handled all Cloud Run
3. Scaffolded entire project structure

## Key Lessons Learned

1. Always check the output of AI, Review the code it generate.
2. sklearn.Pipeline` is worth the setup cost:
   one artifact, zero preprocessing bugs at inference time

## Future Improvements

1. Add SHAP explanations in the UI
2. More features: operating hours, photo count, etc
3. CI/CD with GitHub Actions
4. Map view with prediction heatmap